

FastAPI Guide

Introduction

FastAPI is a modern, high-performance Python web framework for building APIs, based on standard Python type hints. It is built on top of Starlette for the web parts and Pydantic for the data parts, which gives it automatic request validation, serialization, and interactive documentation out of the box.

Because FastAPI generates an OpenAPI schema automatically, every route is instantly documented and testable through the built-in Swagger UI at `/docs` and the ReDoc interface at `/redoc`, with no extra configuration required from the developer.

Why Use FastAPI

- Performance on par with NodeJS and Go, thanks to Starlette and asynchronous support.
- Automatic data validation and serialization using Pydantic models.
- Editor support with type hints, including autocompletion and inline error checking.
- Interactive API documentation generated automatically from code.
- Native support for asynchronous request handlers using `async/await`.

Getting Started

A minimal FastAPI application only requires creating an instance of the FastAPI class and defining path operations with decorators such as `@app.get` or `@app.post`. Each path operation function can declare parameters as normal Python function arguments, and FastAPI infers whether they come from the path, the query string, or the request body based on type hints and default values.

The framework is typically run with an ASGI server such as Uvicorn, which can be launched directly from the command line or embedded in a Python script for local development with automatic reload.

Request and Response Models

Pydantic models describe the shape of incoming and outgoing data. Declaring a model as a path operation parameter tells FastAPI to parse the request body as JSON, validate every field against the declared types, and return a clear, structured error response if validation fails.

Response models can be declared separately using the `response_model` parameter, which lets an endpoint accept a broader internal object while only exposing a specific subset of fields to the client, which is useful for hiding sensitive fields such as password hashes.

Dependency Injection

FastAPI includes a dependency injection system built around the Depends function. Dependencies are plain callables that can validate authentication tokens, open and close database sessions, or enforce shared business rules, and they can themselves depend on other dependencies, which keeps cross-cutting logic out of individual route handlers.

Common Architecture Pattern

Many production FastAPI projects follow a layered structure that separates concerns: a Controller layer defines the HTTP routes and handles request and response objects, a Service layer contains the business logic, and a Filter or middleware layer handles cross-cutting concerns such as authentication, logging, and PII masking before a request reaches the controller.

This separation keeps route handlers thin, makes business logic reusable and easier to unit test, and allows middleware such as authentication or masking filters to be swapped without touching the core application logic.

Error Handling and Middleware

FastAPI allows custom exception handlers to be registered for specific exception classes, so that errors raised anywhere in the application can be converted into consistent, well-structured JSON responses. Middleware functions wrap every request and response, making them a natural place to add logging, CORS headers, or request timing.

Security Basics

FastAPI provides built-in utilities for common authentication schemes, including OAuth2 password flows and JWT bearer tokens, through the `fastapi.security` module. Combined with dependency injection, a single reusable dependency can protect any number of routes by validating a token and returning the current authenticated user.